

Contents

- [Start computational time](#)
- [Read Image](#)
- [Histogram](#)
- [Calculate probabilities](#)
- [Sort according to decreasing probability](#)
- [Generate Shannon-Fano codes](#)
- [Display Table](#)
- [Code Length](#)
- [Average Code Length](#)
- [Entropy](#)
- [Original Bits](#)
- [Compressed Bits](#)
- [Compression Ratio](#)
- [Percentage Compression](#)
- [Redundancy](#)
- [Display Computational Results](#)
- [Execution Time](#)

```
clc;  
clear;  
close all;
```

Start computational time

```
tic;
```

Read Image

```
img = imread('wheat.jpg');  
  
if size(img,3) == 3  
    img = rgb2gray(img);  
end  
  
[rows, cols] = size(img);  
totalPixels = numel(img);  
  
fprintf('          SHANNON-FANO IMAGE COMPRESSION\n');  
fprintf('\nImage Information:\n');  
fprintf('Image Size      : %d x %d pixels\n', rows, cols);  
fprintf('Total Pixels    : %d\n', totalPixels);  
fprintf('Bits per Pixel   : 8\n');  
fprintf('Original Size    : %d bits\n', totalPixels * 8);
```

SHANNON-FANO IMAGE COMPRESSION

Image Information:

Image Size : 914 x 1280 pixels
Total Pixels : 1169920
Bits per Pixel : 8
Original Size : 9359360 bits

Histogram

```
[counts, grayLevels] = imhist(img);  
  
% Remove zero-frequency gray levels  
idx = counts > 0;  
symbols = grayLevels(idx);  
freq = counts(idx);
```

Calculate probabilities

```
prob = freq / totalPixels;
```

Sort according to decreasing probability

```
[prob, order] = sort(prob, 'descend');  
  
symbols = symbols(order);  
freq = freq(order);
```

Generate Shannon-Fano codes

```
codes = cell(length(symbols),1);  
codes = shannonFano(prob, codes, 1, length(symbols));
```

Display Table

```
fprintf('\nGray Level | Frequency | Probability | Code\n');  
for i = 1:length(symbols)  
  
    fprintf('%10d %9d %11.5f %s\n', ...  
        symbols(i), freq(i), prob(i), codes{i});  
  
end
```

Gray Level	Frequency	Probability	Code
195	15897	0.01359	000000
194	15633	0.01336	000001
193	15369	0.01314	000010

193	15368	0.01314	000010
196	15236	0.01302	0000110
197	15047	0.01286	0000111
192	14976	0.01280	000100
198	14622	0.01250	000101
190	14597	0.01248	000110
191	14583	0.01246	0001110
189	14479	0.01238	0001111
188	14388	0.01230	001000
187	14128	0.01208	001001
199	14105	0.01206	001010
186	13983	0.01195	0010110
185	13961	0.01193	0010111
184	13769	0.01177	001100
182	13559	0.01159	001101
183	13396	0.01145	001110
181	13236	0.01131	0011110
200	13178	0.01126	0011111
180	12883	0.01101	010000
201	12787	0.01093	0100010
179	12661	0.01082	0100011
177	12517	0.01070	010010
178	12512	0.01069	0100110
176	12359	0.01056	0100111
175	12246	0.01047	010100
174	12095	0.01034	0101010
202	12094	0.01034	0101011
173	11797	0.01008	010110
171	11565	0.00989	0101110
172	11550	0.00987	0101111
203	11366	0.00972	011000
170	11220	0.00959	0110010
169	10969	0.00938	0110011
204	10705	0.00915	0110100
168	10660	0.00911	0110101
167	10340	0.00884	0110110
166	10334	0.00883	0110111
205	10047	0.00859	0111000
165	9954	0.00851	0111001
164	9637	0.00824	0111010
163	9613	0.00822	0111011
206	9226	0.00789	0111100
162	9212	0.00787	0111101
159	8994	0.00769	0111110
160	8972	0.00767	0111111
161	8924	0.00763	1000000
158	8679	0.00742	1000001
207	8676	0.00742	1000010
157	8645	0.00739	1000011
156	8485	0.00725	1000100
155	8139	0.00696	1000101
154	7978	0.00682	1000110
208	7803	0.00667	10001110
153	7794	0.00666	10001111
152	7631	0.00652	1001000
150	7367	0.00630	1001001
151	7342	0.00628	1001010
209	7252	0.00620	10010110

209	7252	0.00020	10010110
149	6963	0.00595	10010111
148	6874	0.00588	1001100
147	6766	0.00578	1001101
146	6512	0.00557	1001110
210	6464	0.00553	10011110
145	6411	0.00548	10011111
144	6155	0.00526	1010000
141	6017	0.00514	10100010
143	6008	0.00514	10100011
142	5864	0.00501	1010010
211	5852	0.00500	10100110
140	5668	0.00484	10100111
139	5564	0.00476	1010100
138	5401	0.00462	10101010
137	5358	0.00458	10101011
212	5247	0.00448	10101100
136	5221	0.00446	10101101
1	5068	0.00433	10101110
135	5039	0.00431	10101111
213	4895	0.00418	10110000
134	4887	0.00418	10110001
133	4779	0.00408	10110010
132	4739	0.00405	10110011
130	4580	0.00391	10110100
131	4575	0.00391	10110101
2	4449	0.00380	10110110
129	4392	0.00375	10110111
128	4316	0.00369	10111000
127	4296	0.00367	10111001
126	4194	0.00358	10111010
125	4092	0.00350	10111011
214	4085	0.00349	10111100
123	3976	0.00340	10111101
124	3952	0.00338	10111110
122	3794	0.00324	101111110
215	3787	0.00324	101111111
4	3702	0.00316	11000000
121	3680	0.00315	11000001
120	3670	0.00314	11000010
119	3667	0.00313	110000110
118	3616	0.00309	110000111
3	3585	0.00306	11000100
116	3552	0.00304	11000101
114	3545	0.00303	11000110
117	3536	0.00302	110001110
115	3442	0.00294	110001111
113	3417	0.00292	11001000
111	3382	0.00289	110010010
112	3353	0.00287	110010011
216	3344	0.00286	11001010
110	3285	0.00281	110010110
109	3232	0.00276	110010111
108	3168	0.00271	11001100
5	3147	0.00269	110011010
217	3047	0.00260	110011011
107	3024	0.00258	11001110
7	2957	0.00253	110011110

106	2937	0.00251	110011111
105	2923	0.00250	11010000
104	2895	0.00247	110100010
103	2856	0.00244	110100011
6	2804	0.00240	11010010
100	2794	0.00239	110100110
102	2784	0.00238	110100111
99	2729	0.00233	11010100
101	2719	0.00232	110101010
98	2676	0.00229	110101011
97	2665	0.00228	110101100
218	2644	0.00226	110101101
13	2630	0.00225	110101110
94	2609	0.00223	110101111
96	2587	0.00221	11011000
93	2568	0.00220	110110010
91	2541	0.00217	110110011
95	2537	0.00217	110110100
92	2536	0.00217	110110101
90	2483	0.00212	110110110
10	2480	0.00212	110110111
89	2442	0.00209	110111000
8	2433	0.00208	110111001
16	2412	0.00206	110111010
12	2361	0.00202	110111011
9	2360	0.00202	110111100
85	2352	0.00201	110111101
88	2339	0.00200	110111110
86	2332	0.00199	110111111
11	2299	0.00197	111000000
219	2297	0.00196	111000001
87	2292	0.00196	111000010
14	2290	0.00196	111000011
84	2279	0.00195	111000100
15	2271	0.00194	111000101
79	2224	0.00190	111000110
83	2221	0.00190	111000111
82	2215	0.00189	111001000
17	2156	0.00184	111001001
81	2140	0.00183	111001010
75	2119	0.00181	111001011
80	2112	0.00181	111001100
77	2105	0.00180	111001101
76	2089	0.00179	111001110
73	2082	0.00178	1110011110
18	2070	0.00177	1110011111
74	2065	0.00177	111010000
78	2063	0.00176	111010001
71	2033	0.00174	111010010
68	2013	0.00172	111010011
69	1979	0.00169	111010100
72	1976	0.00169	111010101
220	1952	0.00167	111010110
70	1938	0.00166	1110101110
19	1935	0.00165	1110101111
63	1923	0.00164	111011000
66	1892	0.00162	111011001

67	1857	0.00159	111011010
60	1837	0.00157	1110110110
65	1833	0.00157	1110110111
21	1829	0.00156	111011100
62	1827	0.00156	111011101
61	1819	0.00155	111011110
221	1814	0.00155	1110111110
64	1808	0.00155	1110111111
57	1781	0.00152	111100000
58	1781	0.00152	111100001
20	1765	0.00151	111100010
56	1740	0.00149	1111000110
59	1733	0.00148	1111000111
55	1717	0.00147	111100100
47	1709	0.00146	111100101
50	1688	0.00144	111100110
53	1677	0.00143	1111001110
52	1676	0.00143	1111001111
54	1658	0.00142	111101000
22	1648	0.00141	1111010010
51	1613	0.00138	1111010011
48	1593	0.00136	111101010
49	1555	0.00133	1111010110
45	1550	0.00132	1111010111
222	1550	0.00132	111101100
42	1513	0.00129	1111011010
23	1498	0.00128	1111011011
44	1494	0.00128	111101110
43	1471	0.00126	1111011110
24	1459	0.00125	1111011111
37	1450	0.00124	111110000
40	1435	0.00123	1111100010
223	1398	0.00119	1111100011
26	1392	0.00119	111110010
39	1391	0.00119	1111100110
41	1385	0.00118	1111100111
46	1385	0.00118	111110100
0	1384	0.00118	1111101010
35	1366	0.00117	1111101011
25	1350	0.00115	1111101100
38	1350	0.00115	1111101101
29	1346	0.00115	1111101110
32	1329	0.00114	1111101111
27	1325	0.00113	111111000
34	1314	0.00112	1111110010
36	1296	0.00111	1111110011
33	1254	0.00107	1111110100
31	1250	0.00107	1111110101
30	1219	0.00104	1111110110
224	1176	0.00101	1111110111
28	1166	0.00100	1111111000
225	1060	0.00091	1111111001
254	931	0.00080	1111111010
226	880	0.00075	11111110110
227	736	0.00063	11111110111
228	567	0.00048	11111111000
229	530	0.00045	11111111001

230	403	0.00034	11111111010
253	383	0.00033	111111110110
231	302	0.00026	111111110111
232	273	0.00023	111111111000
252	230	0.00020	111111111001
233	201	0.00017	1111111110100
234	189	0.00016	1111111110101
251	146	0.00012	1111111110110
235	142	0.00012	1111111110111
236	116	0.00010	1111111111000
250	116	0.00010	11111111110010
237	97	0.00008	11111111110011
239	97	0.00008	1111111111010
249	96	0.00008	11111111110110
238	94	0.00008	11111111110111
240	82	0.00007	11111111111000
241	72	0.00006	11111111111001
242	72	0.00006	11111111111010
243	69	0.00006	11111111111011
245	69	0.00006	11111111111100
247	69	0.00006	111111111111010
248	64	0.00005	111111111111011
246	59	0.00005	11111111111110
244	57	0.00005	111111111111110
255	44	0.00004	111111111111111

Code Length

```
codeLength = cellfun(@length, codes);
```

Average Code Length

```
avgLength = sum(prob .* codeLength);
```

Entropy

```
entropyValue = -sum(prob .* log2(prob));
```

Original Bits

```
originalBits = totalPixels * 8;
```

Compressed Bits

```
compressedBits = sum(freq .* codeLength);
```

Compression Ratio

```
compressionRatio = originalBits / compressedBits;
```

Percentage Compression

```
compressionPercentage = ...  
    (1 - compressedBits/originalBits) * 100;
```

Redundancy

```
redundancy = avgLength - entropyValue;
```

Display Computational Results

```
fprintf('\n          COMPUTATIONAL RESULTS\n');  
  
fprintf('\nNumber of Symbols      : %d\n', length(symbols));  
  
fprintf('Entropy                    : %.5f bits/pixel\n', ...  
    entropyValue);  
  
fprintf('Average Code Length      : %.5f bits/pixel\n', ...  
    avgLength);  
  
fprintf('Original Bits            : %d bits\n', ...  
    originalBits);  
  
fprintf('Compressed Bits          : %d bits\n', ...  
    compressedBits);  
  
fprintf('Bits Saved               : %d bits\n', ...  
    originalBits - compressedBits);  
  
fprintf('Compression Ratio        : %.5f\n', ...  
    compressionRatio);  
  
fprintf('Compression Percentage   : %.2f %%\n', ...  
    compressionPercentage);  
  
fprintf('Coding Redundancy        : %.5f bits/pixel\n', ...  
    redundancy);
```

COMPUTATIONAL RESULTS

```
Number of Symbols      : 256  
Entropy                : 7.42171 bits/pixel  
Average Code Length    : 7.47928 bits/pixel  
Original Bits          : 9359360 bits  
Compressed Bits        : 8750160 bits  
Bits Saved             : 609200 bits  
Compression Ratio      : 1.06962  
Compression Percentage  : 6.51 %  
  
Coding Redundancy      : 0.05757 bits/pixel
```

Execution Time

```
executionTime = toc;

fprintf('\nExecution Time           : %.6f seconds\n', ...
    executionTime);

function codes = shannonFano(prob, codes, first, last)

    % If only one symbol is present
    if first == last

        if isempty(codes{first})
            codes{first} = '0';
        end

        return;
    end

    % Total probability of current group
    total = sum(prob(first:last));

    cumulative = 0;
    minDiff = inf;
    split = first;

    % Find division point closest to half
    for i = first:last-1

        cumulative = cumulative + prob(i);

        difference = abs(total/2 - cumulative);

        if difference < minDiff

            minDiff = difference;
            split = i;
        end
    end

    % Assign 0 to first group
    for i = first:split
        codes{i} = [codes{i} '0'];
    end

    % Assign 1 to second group
    for i = split+1:last
        codes{i} = [codes{i} '1'];
    end

    % Recursively divide first group
    codes = shannonFano(prob, codes, first, split);

    % Recursively divide second group
```

```
% recursively divide second group
```

```
codes = shannonFano(prob, codes, split+1, last);
```

```
end
```

Execution Time : 0.122828 seconds

Published with MATLAB® R2018a